

Assignment 2: Policy Gradient

Andrew ID: ncheng2

Collaborators: None.

NOTE: Please do NOT change the sizes of the answer blocks or plots.

5 Small-Scale Experiments (7.5 Pts)

5.1 Experiment 1 (Cartpole) – [4.5 points total]

5.1.1 Configurations

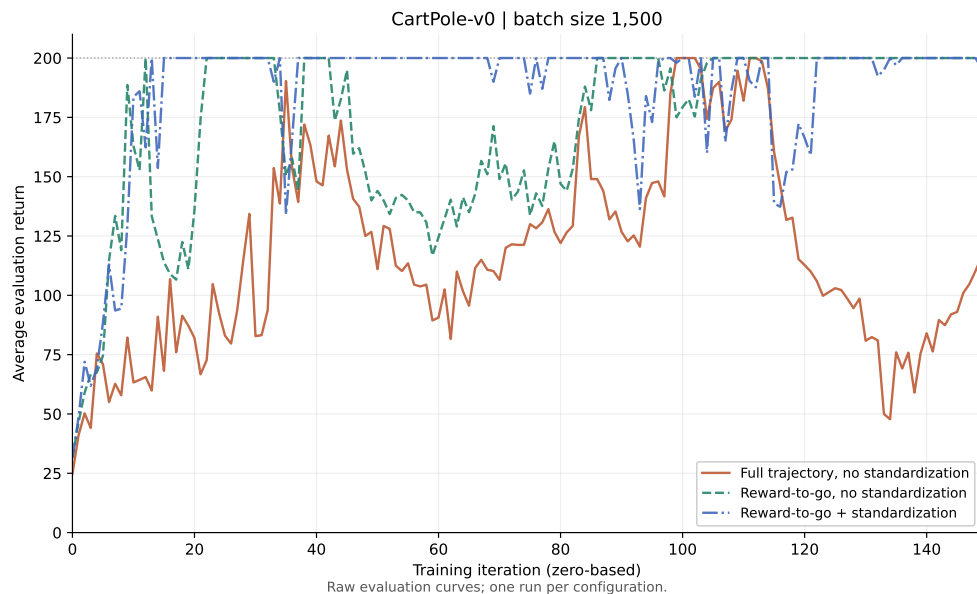
Q5.1.1

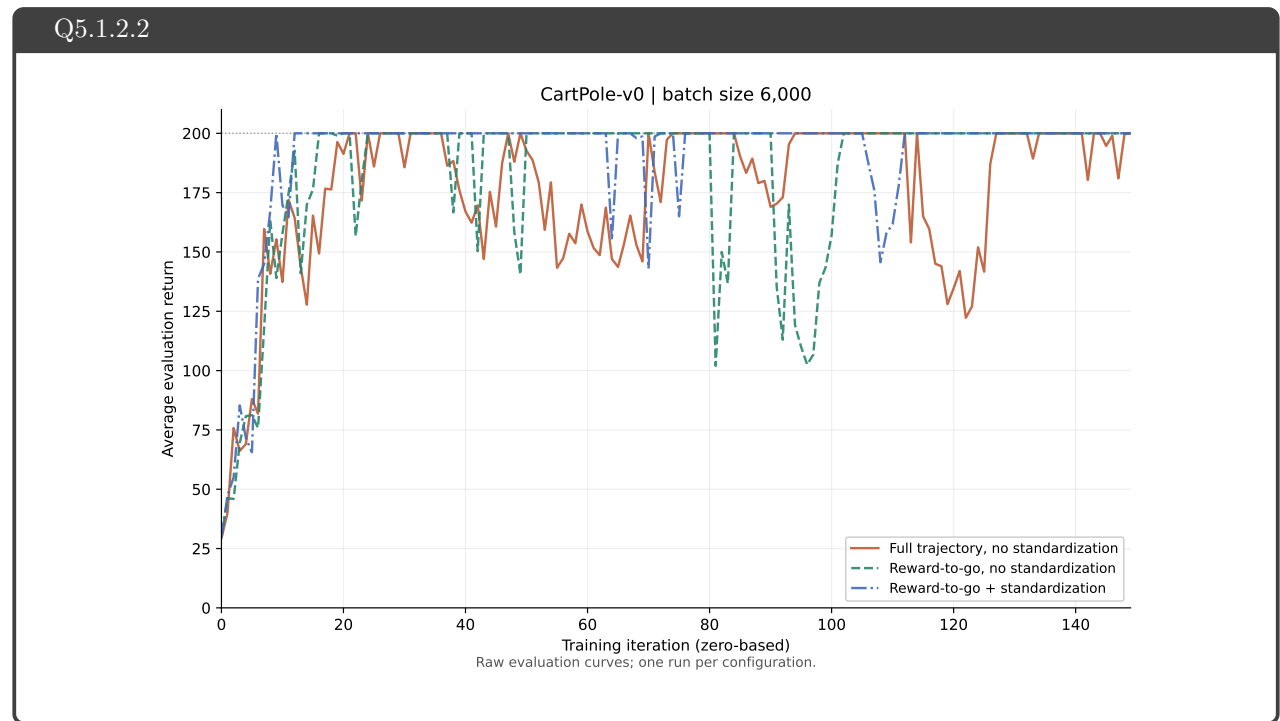
```
python -m rob831.scripts.run_hw2 --env_name CartPole-v0 -n 150 -b 1500 \
-dsa --exp_name q1_sb_no_rtg_dsa
python -m rob831.scripts.run_hw2 --env_name CartPole-v0 -n 150 -b 1500 \
-rtg -dsa --exp_name q1_sb_rtg_dsa
python -m rob831.scripts.run_hw2 --env_name CartPole-v0 -n 150 -b 1500 \
-rtg --exp_name q1_sb_rtg_na
python -m rob831.scripts.run_hw2 --env_name CartPole-v0 -n 150 -b 6000 \
-dsa --exp_name q1_lb_no_rtg_dsa
python -m rob831.scripts.run_hw2 --env_name CartPole-v0 -n 150 -b 6000 \
-rtg -dsa --exp_name q1_lb_rtg_dsa
python -m rob831.scripts.run_hw2 --env_name CartPole-v0 -n 150 -b 6000 \
-rtg --exp_name q1_lb_rtg_na
```

5.1.2 Plots

5.1.2.1 Small batch – [1 points]

Q5.1.2.1



5.1.2.2 Large batch – [0.5 points]**5.1.3 Analysis****5.1.3.1 Value estimator – [1 points]****Q5.1.3.1**

Without advantage standardization, reward-to-go performed better, especially with the small batch. The mean evaluation returns over the last 20 iterations were 82.60 (full trajectory) versus 200.00 (reward-to-go) for batch size 1500, and 197.20 versus 200.00 for batch size 6000. Reward-to-go excludes rewards received before the action and thus removes irrelevant past-reward terms from the gradient estimator, reducing its variance.

5.1.3.2 Advantage standardization – [1 points]**Q5.1.3.2**

Standardization improved early learning in these runs but final performance was similar. Centering and scaling advantages makes updates less sensitive to the return scale.

5.1.3.3 Batch size – [1 points]

Q5.1.3.3

Increasing the batch size from 1500 to 6000 helped the full-trajectory estimator most. Larger batches average over more trajectories and can reduce sampling noise, but require more environment steps and time per update.

5.2 Experiment 2 (InvertedPendulum) – [3 points total]

5.2.1 Configurations – [1 points]

Q5.2.1

```
export OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
# All 15 new search configurations; seed 1 (default).
for spec in "50 0.01" "50 0.02" "50 0.03" "50 0.04" "50 0.05" \
            "100 0.01" "100 0.02" "100 0.03" "100 0.04" "100 0.05" \
            "200 0.01" "200 0.02" "200 0.03" "200 0.1" "500 0.05"; do
    read -r b r <<< "$spec"
    python -m rob831.scripts.run_hw2 --env_name InvertedPendulum-v4 \
        --ep_len 1000 --discount 0.92 -n 100 -l 2 -s 64 \
        -b "$b" -lr "$r" -rtg --no_gpu --exp_name "q2_b${b}_r${r}"
done
```

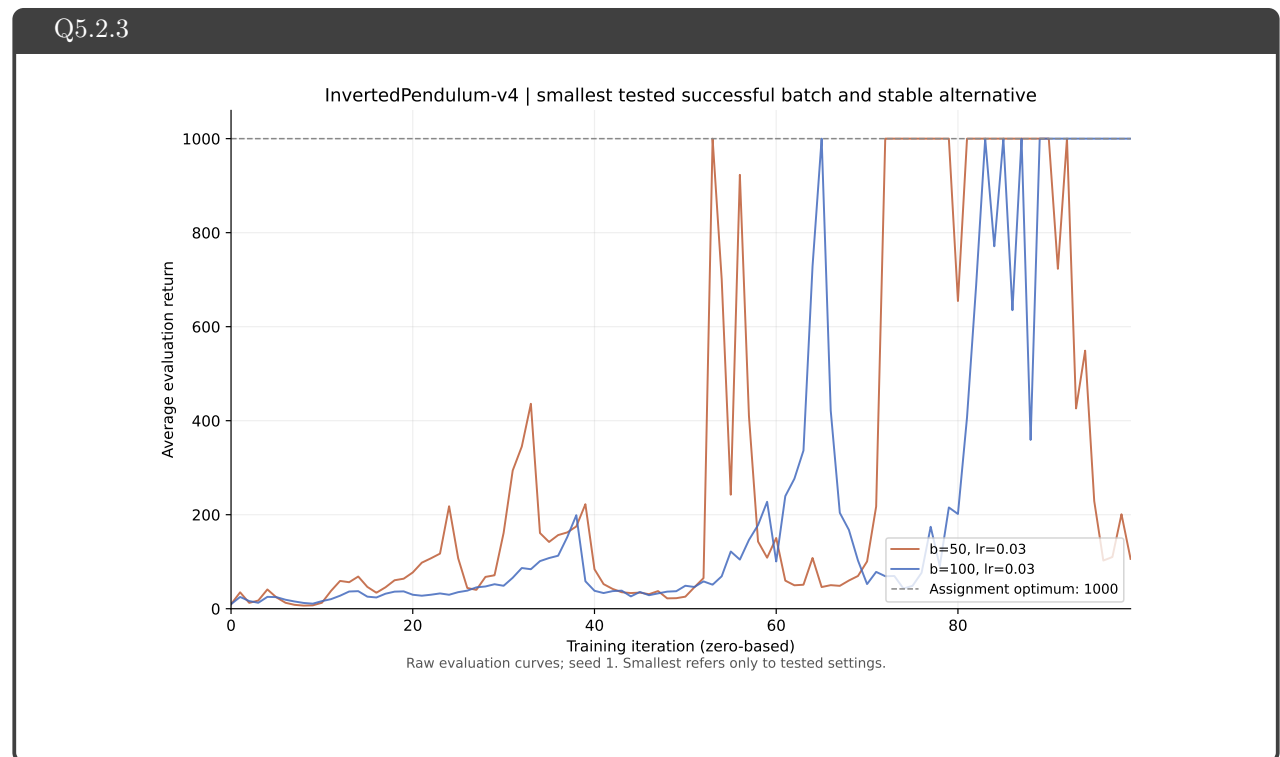
Two pre-existing runs ($b = 500, 1000$, $r = 0.01$) were also inspected. New runs used the activated homework Conda environment and CPU execution; all other arguments retain their defaults (seed 1, evaluation batch 400, video logging disabled). All search runs completed 100 iterations.

5.2.2 smallest b^* and largest r^* (same run) – [1 points]

Q5.2.2

Among tested configurations, the smallest batch reaching 1000 was $b^* = 50$; its largest tested successful rate was $r^* = 0.03$ in the same run. It first reached 1000 at iteration 53 (zero-based), with 20 evaluations at 1000, but its final return was 106 and last-10 mean was 444.65. For a more stable alternative, $b = 100, r = 0.03$ first reached 1000 at iteration 65 and stayed at 1000 for the final 10 iterations. Both curves are shown below. This finite, single-seed search establishes tested outcomes, not global extrema; a batch target is a minimum because complete trajectories are collected.

5.2.3 Plot – [1 points]



7 More Complex Experiments (4.5 Pts)

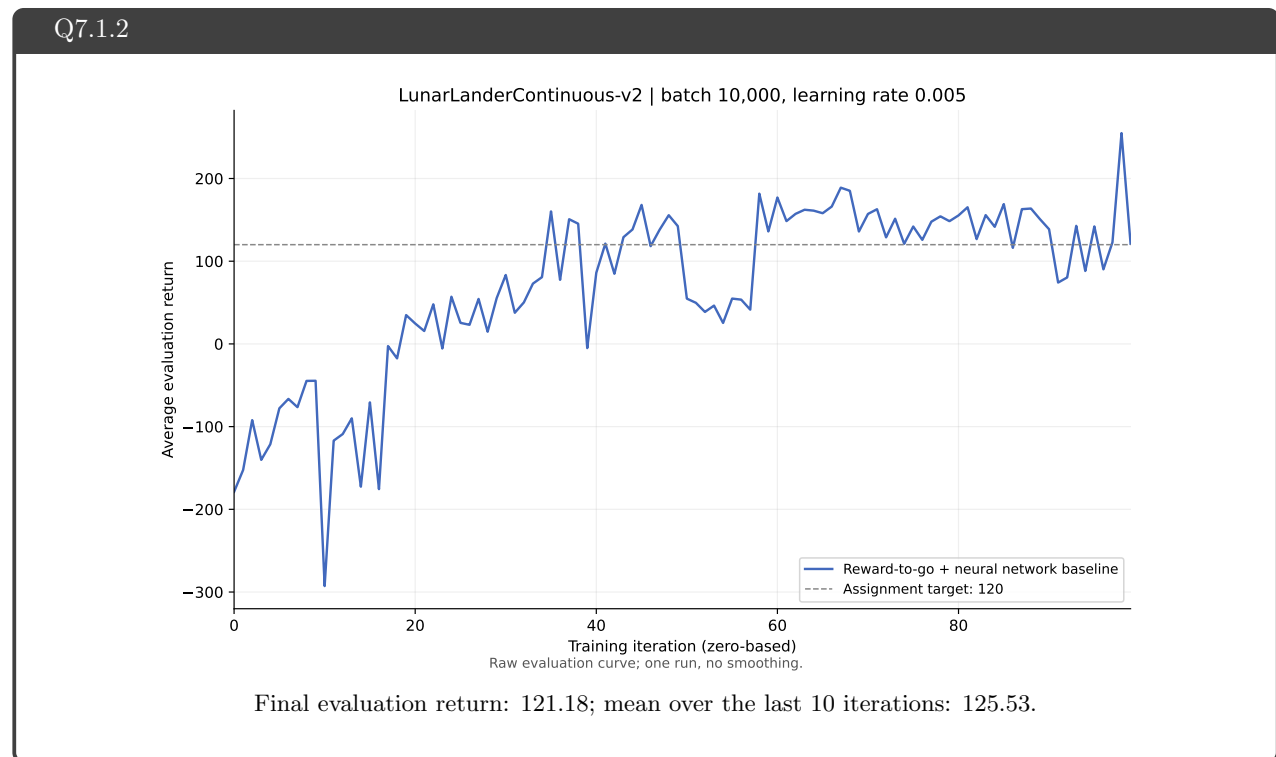
7.1 Experiment 3 (LunarLander) – [1 points total]

7.1.1 Configurations

Q7.1.1

```
python -m rob831.scripts.run_hw2 \  
  --env_name LunarLanderContinuous-v2 --ep_len 1000 \  
  --discount 0.99 -n 100 -l 2 -s 64 -b 10000 -lr 0.005 \  
  --reward_to_go --nn_baseline --exp_name q3_b10000_r0.005
```

7.1.2 Plot – [1 points]



7.2 Experiment 4 (HalfCheetah) – [3.5 points]

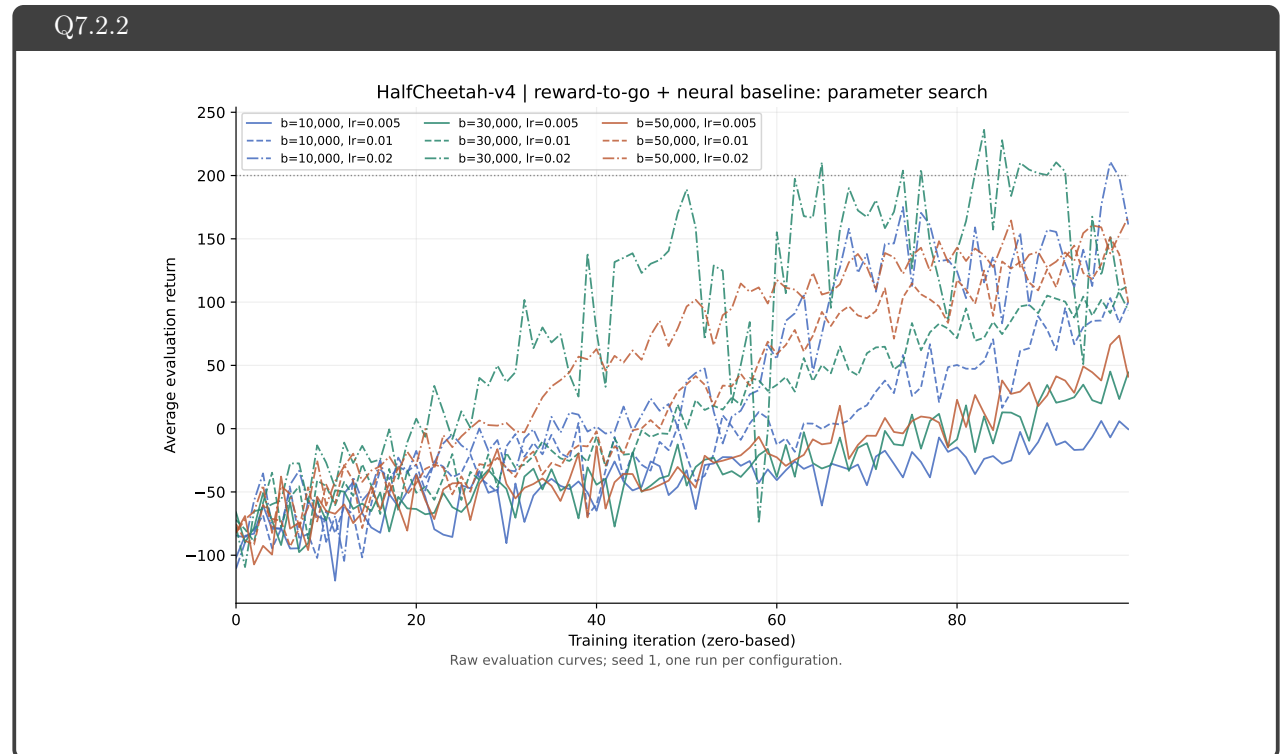
7.2.1 Configurations

Q7.2.1

```
# CPU runs, seed 1 (default); one thread per process.
export OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
for b in 10000 30000 50000; do
  for r in 0.005 0.01 0.02; do
    python -m rob831.scripts.run_hw2 --env_name HalfCheetah-v4 \
      --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 \
      -b "$b" -lr "$r" -rtg --nn_baseline --no_gpu \
      --exp_name "q4_search_b${b}_lr${r}_rtg_nnbaseline"
  done
done
```

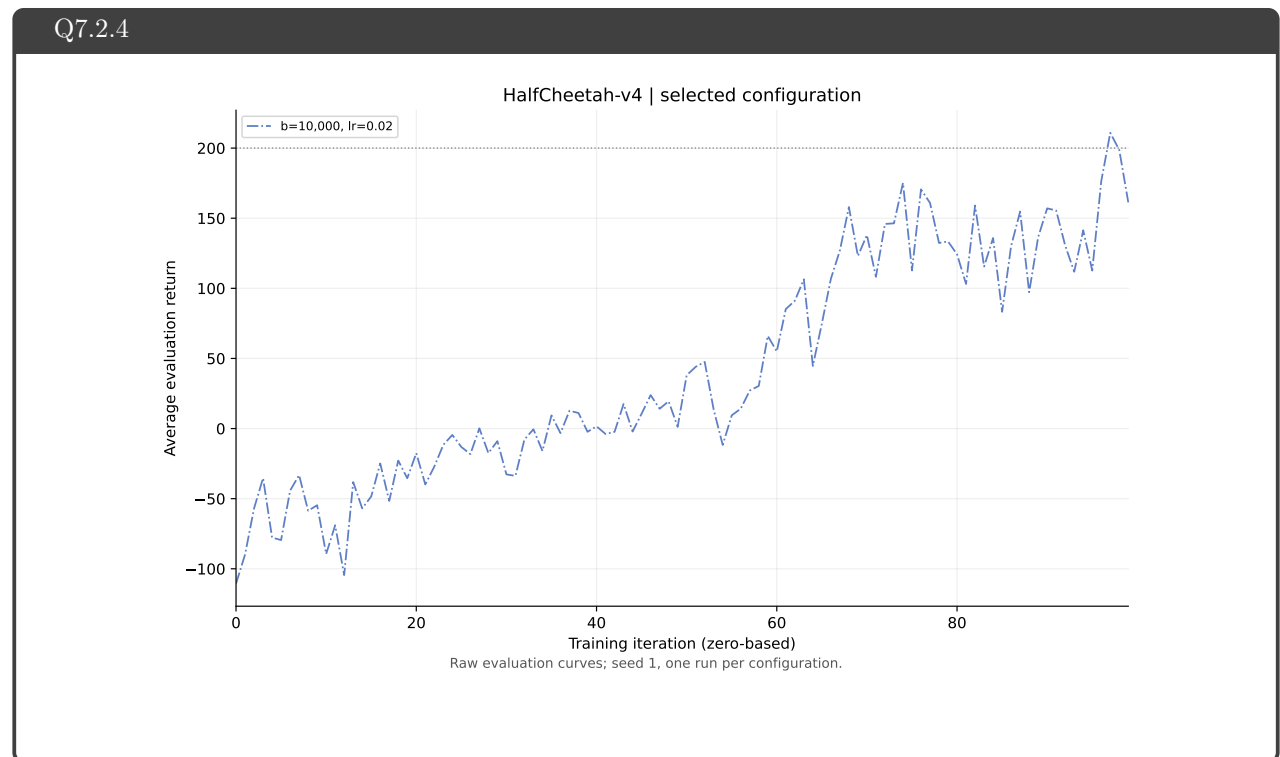
All nine combinations were run. The selected pair maximizes the mean raw evaluation return over the last 10 iterations; each configuration uses one seed. The loops list the commands; jobs were executed concurrently.

7.2.2 Plot – [1 points]

7.2.3 Optimal b^* and r^* – [0.5 points]

Q7.2.3

The selected pair is $b^* = 10000$ and $r^* = 0.02$. It has the highest mean evaluation return over the last 10 iterations (155.60) among the nine tested configurations, compared with 141.57 for $b = 30000, r = 0.02$ and 146.41 for $b = 50000, r = 0.02$. Its peak evaluation return is 210.90 and final return is 161.29; it does not maintain the target of 200 by the end of this seed-1 run.

7.2.4 Plot – [0.5 points]**7.2.5 Describe how b^* and r^* affect task performance – [0.5 points]****Q7.2.5**

Increasing the learning rate improved the last-10 mean at every batch size. For $b = 10000$, rates 0.005, 0.01, and 0.02 gave -5.37 , 83.98 , and 155.60 ; thus 0.005 learned too slowly over these 100 updates. Larger batches helped at smaller rates: at $r = 0.01$, the means increased from 83.98 to 100.46 to 127.74 as b increased. However, at $r = 0.02$, the means were 155.60 , 141.57 , and 146.41 , so larger batches did not improve the selection metric. More samples can reduce gradient noise but cost more environment interactions per update. These are empirical, single-seed results; larger batches are not guaranteed to improve final return.

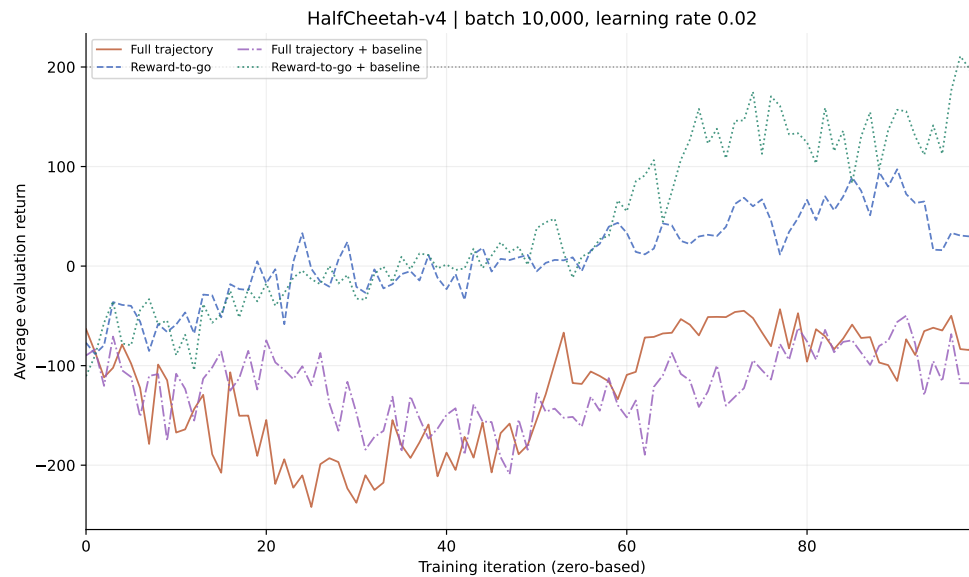
7.2.6 Configurations with optimal b^* and r^* – [0.5 points]

Q7.2.6

```
python -m rob831.scripts.run_hw2 --env_name HalfCheetah-v4 --ep_len 150 \
--discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 --no_gpu \
--exp_name q4_b10000_r0.02
python -m rob831.scripts.run_hw2 --env_name HalfCheetah-v4 --ep_len 150 \
--discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 --no_gpu -rtg \
--exp_name q4_b10000_r0.02_rtg
python -m rob831.scripts.run_hw2 --env_name HalfCheetah-v4 --ep_len 150 \
--discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 --no_gpu --nn_baseline \
--exp_name q4_b10000_r0.02_nnbaseline
python -m rob831.scripts.run_hw2 --env_name HalfCheetah-v4 --ep_len 150 \
--discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 --no_gpu -rtg \
--nn_baseline --exp_name q4_b10000_r0.02_rtg_nnbaseline
```

7.2.7 Plot for four runs with optimal b^* and r^* – [0.5 points]

Q7.2.7



Last-10 means (full trajectory / reward-to-go / baseline / both): -74.79 / 44.50 / -92.78 / 155.60.

8 Implementing Generalized Advantage Estimation (3 Pts)

8.1 Experiment 5 (Hopper) – [3 points]

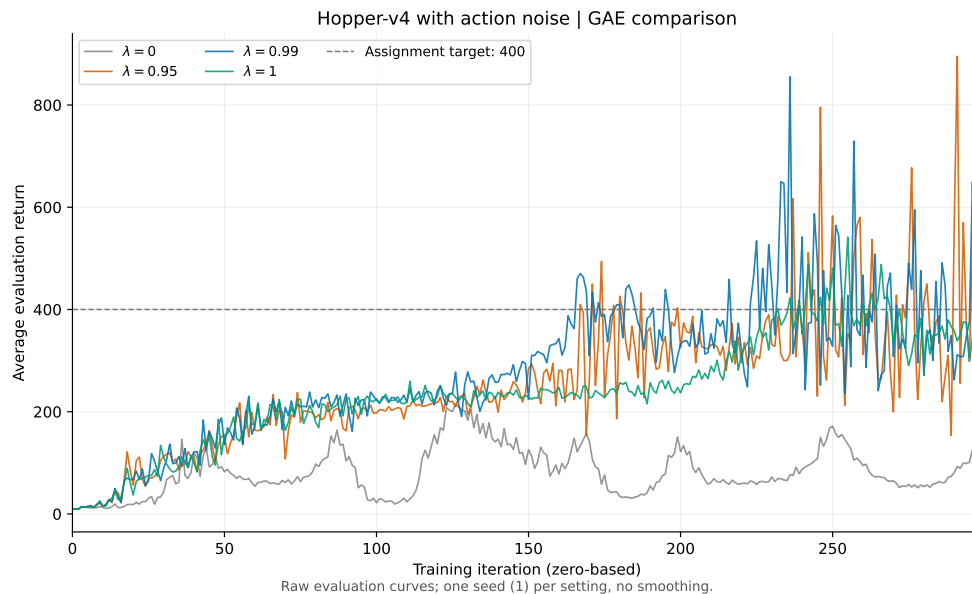
8.1.1 Configurations

Q8.1.1

```
for lam in 0 0.95 0.99 1; do
  python -m rob831.scripts.run_hw2 --env_name Hopper-v4 --ep_len 1000 \
    --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr 0.001 \
    --reward_to_go --nn_baseline --action_noise_std 0.5 --no_gpu \
    --gae_lambda "$lam" --exp_name "q5_b2000_r0.001_lambda${lam}"
done
```

8.1.2 Plot – [1 points]

Q8.1.2



8.1.3 Describe how λ affects task performance – [1.5 points]

Q8.1.3

The last-10-iteration mean evaluation returns for $\lambda = 0, 0.95, 0.99, 1$ were 113.93, 471.31, 396.79, and 359.30, respectively. Thus $\lambda = 0.95$ performed best over this window, while 0.99 had the highest last-50 mean (393.10 versus 386.88 for 0.95). Both intermediate values achieved performance near or above the target of 400. A small λ relies heavily on the learned value function: lower variance, but more bias from value errors. Larger λ uses longer reward sequences, reducing this bias at the cost of higher variance; $\lambda = 1$ recovers the Monte Carlo baseline estimator. These single-seed, noisy curves favor an intermediate tradeoff, with the ranking depending on the evaluation window.

- 8.1.4 Find one robotics paper from the last 5 years that uses policy gradients or an actor-critic method as part of its methodology. In a short paragraph (4–5 sentences): (i) cite the paper, (ii) briefly describe the robot task and how reinforcement learning is used, and (iii) identify one technique from this homework (e.g., reward-to-go, a value-function baseline, or GAE) that is related to the method used in the paper. A few sentences is enough; this is not meant to be an extensive literature review – [0.5 points]**

Q8.1.4

Miki et al., “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics* 7(62), 2022 (<https://arxiv.org/abs/2201.08117>)

The task is to follow velocity commands over challenging terrain using onboard proprioceptive and terrain observations. They train a teacher policy in simulation with Proximal Policy Optimization (PPO), giving it privileged terrain and robot information. A student policy then learns to imitate the teacher from noisy observations and is deployed on the real robot. Their PPO training uses generalized advantage estimation with $\lambda = 0.95$ (Supplementary Table S1), connecting the method to our Hopper experiment and its bias–variance tradeoff in advantage estimation.